Writing NPU quant kernels: SIMT vs PTO vs ASC

A VF code-style and performance comparison of three vector frontends expressing one quantization algorithm.

- 22 kernel variants x 4 implementations = 88 kernels, all passing the same checks
- Simulated VF cycles for all 66 vector-tier kernels
- tilelang 0.1.15, ptoas vmi 0.1.9, CANN 9.2.0-beta.2, SoC `Ascend950PR_9599`
- Single core, `T.Kernel(1)`; the comparison is of the vector body only

All cycle figures are from `msprof op simulator`, not silicon.

One algorithm, three frontends

Group-wise quantization, in PyTorch:

```
grouped = x.float().view(m, k // group_size, group_size)
amax = grouped.abs().amax(dim=-1).clamp(min=E4M3_CLAMP_MIN)
sf = amax / E4M3_MAX
q = (grouped * (E4M3_MAX / amax).unsqueeze(-1)).view(m, k).to(torch.float8_e4m3fn)
```

frontend	what you name	the unit you think in
ASC (`T.simd`, `S`)	the physical instruction	a 64-lane vector register
PTO/VMI (`T.vmi`, `V`)	the intent; width and segments are *arguments*	a logical vector, any width
SIMT (`T.SimtVF`)	one element	a thread

The vector body runs from **6 to 57 lines** depending on frontend and configuration, and the frontends differ by up to **99.62x** in cycles.

The headline: two axes that disagree

It is tempting to assume the frontend writing fewer operations produces the faster kernel. Across these 22 variants, it does not:

PTO relative to ASC	
vector operations written	**74%**
source lines written	**89%**
simulated VF cycles	**0.92x mean**, range 0.64x-1.13x

- PTO uses fewer cycles than ASC on **9 of 22** variants
- On **four** of those it *writes more operations than ASC*

Every pattern in this deck is therefore shown with both axes.

Method, and why the numbers can be trusted

Measurement. Sum the `cycles` column of `core0.veccore0_instr_exe.csv` over rows whose `instr` names a VF launch (`VF`, or `VF_SIMT`). Key on `instr`, never `pipe` – every real VF row is on `PUSHQ`, shared with `MOVEMASK`.

Correctness first. Every kernel is compared against a standalone PyTorch oracle, with an assertion that results came back from the device. No cycles are reported for a kernel that computed the wrong answer.

Like-for-like. VF launch counts compared across tiers on every variant; they match everywhere except `per_token/05` (34/34/32 – SIMT skips a gather setup).

Two failure modes caught: the profiler measuring a `torch.zeros` fill instead of the kernel (caught because three implementations reported *identical* cycles), and divergence inflating one SIMT row (slide 20). Every trace is now audited against the simulator's instruction log.

Pattern 1 – the data path: a per-group scale broadcast

`cast_back/01`: 64 consecutive channels straddle two scale groups, so each lane needs the right one of two scalars.

ASC – build the vector: two broadcast loads, then select by lane mask:

```
lo = S.vld(sf_ub[group], dist="BRC_B32")
hi = S.vld(sf_ub[group + 1], dist="BRC_B32")
scale = S.vsel(lo, hi, mask_low)
```

PTO – state the shape instead: one load, two segments:

```
scale = V.vload(sf_ub[group], size=LANES, stride=1,
                dist_mode="brc", group=groups_per_strip)
```

SIMT – a thread knows which element it owns:

```
for i in T.Parallel(hidden):
    o_l[i] = q_l[i] * sf_ub[i // group_size]
```

Pattern 1 – measured

	ASC	PTO	SIMT
VF-region lines	16	12	**6**
simulated cycles	2,105	2,201	6,908
vs ASC	1.00x	1.05x	3.28x

This row sets the theme.

- PTO collapsed three operations into one and came out **slower**
- SIMT wrote a third of the code and cost **3.28x**
- Neither axis predicts the other

Same variant, quieter win: switching output dtype costs ASC a different *store instruction*, PTO only a convert, and SIMT **nothing at all** – `T.copy` converts between buffer dtypes, so the VF body never mentions it.

Pattern 2 – where SIMT genuinely wins: an interleave

`per_channel/02` packs two token groups' UE8M0 bytes into each 16-bit word: byte `2c` from group `2i`, byte `2c+1` from group `2i+1`.

ASC / PTO – that is an interleave instruction:

```
a = S.vld(sf_rows_ub[0, col], dist="NORM_B8")
b = S.vld(sf_rows_ub[1, col], dist="NORM_B8")
# A uint8 register is 256 lanes, so ‘lo‘ already holds the
# interleave of the first 128 elements of each input --
```

SIMT – two scalar stores:

```
for c in T.Parallel(hidden):
    packed_ub[PACK_FACTOR * c] = sf_rows_ub[0, c]
    packed_ub[PACK_FACTOR * c + 1] = sf_rows_ub[1, c]
```

Pattern 2 – the trap underneath, and the cost

A `uint8` vector register is **256 lanes**, so loading the 128 meaningful bytes reads 128 bytes past the row. Both SIMD tiers must pad the scratch buffer:

```
sf_rows_ub = T.alloc_shared((PACK_FACTOR, hidden + BYTE_LANES), T.uint8)
```

```
sf_rows_ub = T.alloc_shared((PACK_FACTOR, hidden), T.uint8)
```

	ASC	PTO	SIMT
VF-region lines	35	40	**17**
simulated cycles	849	811	5,964
vs ASC	1.00x	0.96x	7.02x

SIMT removes the instruction *and* the bug it invites – at **7.02x** the cycles. Note PTO is **longer** than ASC here (40 lines vs 35).

Pattern 3 – where SIMT genuinely wins: a transpose

`per_token/05` writes the scale array transposed. The scales live one-per-group in vector lanes, and **you cannot restride a register**.

ASC – compute, for every lane, which source element it should receive:

```
lane = S.vci(0, T.int32)
token_of_lane = S.vand(T.reinterpret(lane, "uint32x64"),
                       S.vdup(token_block - 1, T.uint32))
group_of_lane = T.reinterpret(S.vshrs(lane, log2_block), "uint32x64")
S.vsts(idx_ub[0],
       S.vadd(S.vmuls(token_of_lane, SF_STRIDE), group_of_lane),
       dist="NORM_B32")
```

. . . then gather:

```
S.vsts(sf_out_ub[base, 0],
       S.vgather2(sf_dense_ub[0, base], idx), dist="NORM_B32")
```

SIMT – the transposed layout is the index you store at:

```
sf_out_ub[g, sub] = T.reinterpret("float32", biased << 23)
```

Pattern 3 – the sharpest single trade

	ASC	PTO	SIMT
VF-region lines	54	48	**15**
simulated cycles	5,477	5,448	75,281
vs ASC	1.00x	0.99x	13.74x

- **15 lines against 54** is the largest source-length win in all 22 variants
- **13.74x** is among the worst cycle ratios
- The thread model removes the lane-placement problem, and leaves most of the 128-lane unit idle doing it

PTO is *marginally longer* than ASC on the gather itself: VMI has no scalar-operand `vmul`, so the stride must be broadcast into a vector first.

The rule: SIMT pays off exactly where the SIMD tiers had to move data between lanes – *broadcast, interleave, transpose, gather.*

Pattern 4 – where PTO's SIMD style is already clean enough

`per_channel/01` reduces along **M**. A channel *is* a lane, so there is no cross-lane step for anybody.

ASC – a register accumulator, maxed over 32 rows:

```
acc = S.alloc_local((1,), T.float32)
# seeding with the clamp floor makes the clamp free
acc[0] = S.vdup(E4M3_CLAMP_MIN, T.float32)
for row in T.serial(group_tokens):
    v = S.vabs(S.vcvt(S.vld(x_ub[row, col], dist="UNPK_B16"),
                      T.float32))
```

PTO – the same code with a mask operand:

```
acc = V.alloc_local((1,), V.vreg(LANES, T.float32))
# seeding with the clamp floor makes the clamp free
acc[0] = V.vbrc(T.float32(E4M3_CLAMP_MIN), size=LANES)
for row in T.serial(group_tokens):
    v = V.vabs(V.vcvt(V.vload(x_ub[row, col], size=LANES),
                      "float32"), mask)
```

Pattern 4 – a genuine draw

	ASC	PTO	SIMT
VF-region lines	21	21	**12**
simulated cycles	502	387	2,677
vs ASC	1.00x	0.77x	5.33x

17 vector operations against 16, and 21 source lines each.

- VMI's wins come from expressing *segmented* behaviour and from width being an argument. A reduction that is already one instruction per vector offers neither.
- SIMT's 12 lines buy **notation, not structure** – same three stages, same memory traffic.
- PTO is nonetheless **0.77x** in cycles, so "nothing to buy" is about source.

Second case: `per_block/01` has one scale per 32x32 tile. A scalar has no layout, so a column-major output is free in **all three** frontends – the same configuration that costs ASC a gather in Pattern 3.

Pattern 5 – where PTO's logical style is the real win

`per_token/01` needs the max of each 32-channel group. ASC's vector reduction yields one value per register, so four groups means four masked reductions and four single-element stores:

```
S.vsts(amax_ub[group], S.vcmax(a0, mask_low), dist="ONEPT_B32")
S.vsts(amax_ub[group + 1], S.vcmax(a0, mask_high),
       dist="ONEPT_B32")
S.vsts(amax_ub[group + 2], S.vcmax(a1, mask_low),
       dist="ONEPT_B32")
S.vsts(amax_ub[group + 3], S.vcmax(a1, mask_high),
       dist="ONEPT_B32")
```

PTO says "reduce, in four segments":

```
V.vstore(V.vcmax(V.vabs(x, mask), mask, group=groups_per_pair),
         amax_ub[group])
```

Eight operations become three. The matching broadcast collapses the same way.

Pattern 5 – the one place both axes agree

	ASC	PTO	SIMT
VF-region lines	37	24	**15**
simulated cycles	6,176	4,576	71,606
vs ASC	1.00x	0.74x	11.59x

	ASC	PTO
vector operations	39	**19**

- PTO **halves** the operation count *and* uses **0.74x** the cycles
- Also where SIMT's cost first reaches an order of magnitude, and the code shows why: with `group_size = 32` and `hidden = 128` the reduction runs **4 threads out of 128**

Pattern 5b – width as an argument

`per_token/02` rounds the scale to a power of two by rewriting the float32 exponent field. **ASC** must bake the lane count into every type:

```
one = S.vdup(1, T.uint32)
bits = T.reinterpret(S.vmuls(clamped, inv_qmax), "uint32x64")
# ((bits - 1) >> 23) + 1  ==  ceil(log2(v)) + 127
biased = S.vadds(S.vshrs(S.vsub(bits, one), 23), 1)
sf = T.reinterpret(S.vshls(biased, 23), "float32x64")
# 254 - biased == 127 - ceil_exp: the negated exponent
inv = T.reinterpret(
```

PTO takes the width as a parameter – so this becomes a macro serving 4, 64, 128 or 256 lanes. ASC structurally *cannot* write it once, because `"uint32x64"` fixes the width:

```
one = V.vbrc(T.uint32(1), size=lanes)
shift = V.vbrc(T.uint32(23), size=lanes)
bits = V.vinterpret_cast(
    V.vmul(clamped, V.vbrc(T.float32(inv_qmax), size=lanes), mask), "uint32")
# ((bits - 1) >> 23) + 1  ==  ceil(log2(v)) + 127
biased = V.vadd(V.vshr(V.vsub(bits, one), shift), one)
sf = V.vinterpret_cast(V.vshl(biased, shift), "float32")
```

45 operations against 14 – the largest static gap in the set – at a **1.02x cycle draw**. PTO's real advantage here is **reusability**, not cycles.

Pattern 6 – where PTO costs you

Mandatory width and mask operands. `cast_back/02` decodes a UE8M0 byte. ASC names the distribution mode on the load:

```
packed = S.vld(sf_ub[strip], dist="BRC_B16")
bits = S.vand(S.vshl(T.reinterpret(packed, "uint32x64"),
                     sf_shift), exp_mask)
# An exponent field with a zero mantissa *is* 2^(e-127).
```

PTO needs width, stride, group count and a mask spelled out:

```
packed = V.vload(sf_ub[strip], size=LANES * 2, stride=1,
                 dist_mode="brc", group=1)
bits = V.vand(V.vshl(V.vinterpret_cast(packed, "uint32"),
                     sf_shift), exp_mask)
scale = V.vinterpret_cast(bits, "float32")
```

Operations tie at 12 each; **PTO is 4 lines longer**. Two more:

- **A missing capability:** no per-lane variable *right* shift. `S.vshr(v, shifts)` takes a vector; `V.vshrs` takes a scalar only. *No variant here exercises it* – reported because it is the clearest thing ASC can express and VMI cannot.
- `per_channel/04`**:** PTO's longest region, 51 lines against 46, at a **1.00x** draw.

Pattern 7 – where SIMT loses outright

`per_token/04` quantizes to FP4 (e2m1). ASC and PTO own a hardware `float32 -> e2m1` convert; their only real work is pre-rounding to bfloat16 with a sticky bit:

```
low, high = V.vunzip(x, "uint16")
one = V.vbrc(T.uint16(1), size=lanes)
return V.vinterpret_cast(V.vor(high, V.vmin(low, one)), "bfloat16")
```

No FP4 store is reachable from a `SimtVF` region. SIMT declares its output `uint8` and encodes the format by hand – an extract of 18 lines:

```
sh = T.min(126 - exps, 31)
mant_pre = 0x400000 | (mant >> 1)          # the implicit 1, then make room
low = (1 << sh) - 1                        # the bits about to be shifted out

if branch_free:
    # -(cond) is 0 or all-ones. Both arms are always evaluated, which is
```

It is bit-exact against the PyTorch packer, verified before the kernel ever ran.

Pattern 7 – and half the gap is not SIMT's fault

	ASC	PTO	SIMT
VF-region lines	34	23	**15**
simulated cycles	6,160	4,784	613,632
vs ASC	1.00x	0.78x	99.62x

`per_block/03` is the same loss at 16.77x – a pattern, not a one-off. But the 99.62x **decomposes**: 124,294 instructions against ~37,000, **and** 4.94 cycles per instruction against the rest of the tier's 1.90-2.01.

That second factor is **branch divergence**: `T.if_then_else` lowers to a real branch here, not a predicated select. Respelling three conditionals as mask arithmetic issues **45% more instructions and runs 1.78x fewer cycles** – 345,189, or **56.0x** rather than 99.6x. Both spellings ship, byte-identical.

> ***When the hardware has an instruction for the job, the frontend that can name it wins, and the margin is not close.***

The inversion – static counts mispredict cycles

The `per_block` family: PTO writes **more** operations than ASC on all four, and runs **fewer** cycles on three.

variant	ops ASC	ops PTO	cycles PTO/ASC
`per_block/01`	21	22	**0.85x**
`per_block/02`	27	30	1.02x
`per_block/04`	21	22	**0.65x**
`per_block/05`	29	31	1.02x

ASC reduces a 1024-value tile in **16 steps of 64 lanes**; **PTO** reaches 128 float32 lanes, so it is **8 steps of 128**:

```
for chunk in T.serial(num_chunks):
    v = V.vabs(V.vcvt(V.vload(flat_x[chunk * LANES],
                              size=LANES), "float32"), mask)
    V.vstore(V.vcmax(v, mask, group=1), run_ub[chunk])
```

More operations per step, half as many steps – a static count cannot see a trip count. `cast_back/05` reverses it: PTO's **largest static win** *and* its **worst cycle regression (1.13x)**.

The complete measurement (1/2) – `cast_back`, `per_token`

variant	ops A/P	lines A/P/S	cyc A	cyc P	cyc S	P/A	S/A
`cast_back/01`	9/7	16/12/6	2,105	2,201	6,908	1.05	3.28
`cast_back/02`	12/12	12/16/9	2,201	2,233	7,033	1.01	3.20
`cast_back/03`	8/6	9/10/6	2,080	2,144	6,969	1.03	3.35
`cast_back/04`	6/6	6/7/6	2,016	2,048	6,983	1.02	3.46
`cast_back/05`	16/8	18/11/13	2,296	2,584	10,152	1.13	4.42
`cast_back/06`	14/14	21/22/16	2,272	2,528	9,275	1.11	4.08
`per_token/01`	39/19	37/24/15	6,176	4,576	71,606	0.74	11.59
`per_token/02`	45/14	43/20/17	4,608	4,688	71,008	1.02	15.41
`per_token/03`	44/15	37/21/15	4,608	4,672	74,136	1.01	16.09
`per_token/04`	37/17	34/23/15	6,160	4,784	613,632	0.78	99.62
`per_token/05`	56/38	54/48/15	5,477	5,448	75,281	0.99	13.74
`per_token/06`	39/30	57/44/32	6,864	4,576	71,606	0.67	10.43
`per_token/07`	31/17	39/24/15	4,864	5,136	72,490	1.06	14.90

`cast_back` is the data path alone; `per_token` the segmented reduction.

The complete measurement (2/2) – `per_block`, `per_channel`

variant	ops A/P	lines A/P/S	cyc A	cyc P	cyc S	P/A	S/A
`per_block/01`	21/22	21/25/18	1,086	924	19,276	0.85	17.75
`per_block/02`	27/30	28/33/20	918	936	19,444	1.02	21.18
`per_block/03`	28/26	29/29/19	1,625	1,040	27,256	0.64	16.77
`per_block/04`	21/22	21/25/18	1,414	924	19,272	0.65	13.63
`per_block/05`	29/31	33/37/28	918	940	19,444	1.02	21.18
`per_channel/01`	17/16	21/21/12	502	387	2,677	0.77	5.33
`per_channel/02`	28/29	35/40/17	849	811	5,964	0.96	7.02
`per_channel/03`	23/19	29/35/14	930	721	4,073	0.78	4.38
`per_channel/04`	32/33	46/51/17	738	736	5,964	1.00	8.08

`per_block` is the static/dynamic inversion. `per_channel` reduces along M, where the data already lines up with the lanes.

Aggregates

	PTO vs ASC	SIMT vs ASC
vector operations	74%	– (no comparable count)
source lines	89%	**53%**
simulated cycles	**0.92x mean** (0.64x-1.13x)	10.44x mean (3.20x-21.18x)

- PTO faster than ASC on **9 of 22**; ASC faster on 6; 7 within 2%
- SIMT range **excludes** `per_token/04`, whose hand-written FP4 encoder measures a missing instruction, not a programming style
- SIMT writes **53%** of ASC's lines and costs roughly an order of magnitude

Caveats

1. **Simulator cycles, not silicon.** No hardware-fidelity claim.
2. **One shape per variant** (mostly `M=32`, `K=128`). Ratios are within-variant and valid; cycle counts are **not** comparable between rows.
3. **Single core**, no multi-core scheduling, double buffering or cache hints.
4. **The SIMT kernels are deliberately untuned** – written the way a GPU programmer writes them first. A separate suite sweeping SIMT strategies found **0.33x to 53x**, i.e. SIMT *winning* by 3x on some grouped reductions. **Nothing here says "SIMT is slow" in general.**
5. **Line counts exclude helpers outside the vector region**, in every frontend – understating SIMT by ~18 lines on the two FP4 variants.
6. **All reductions use** `group_size = 32`, the case that leaves the most SIMT threads idle.

Conclusions – the two questions answered

Where does SIMT genuinely simplify the kernel?

Only where the SIMD frontends had to move data *between lanes*: a group broadcast, an interleave, a transpose. There it is a third of the code and removes a class of bug – the 256-lane padding trap being the clearest example. Everywhere else it buys notation rather than structure, and it pays an order of magnitude in cycles throughout.

Where is PTO's SIMD style already clean enough?

When the operation maps onto the hardware as written: the data already lines up with the lanes (`per_channel/01` – a channel *is* a lane), or the value being placed is a scalar (`per_block/01` – a tile scale has no layout). Those come out 17 operations against 16, and 21 lines each. Reaching for a thread model there is effort for nothing.

Guidance, and one methodological point

	choose it when
SIMT	reaching a correct kernel quickly matters more than cycles; or the kernel is dominated by lane placement – packing, transposes, gathers
PTO/VMI	the default for production vector code: 74% of ASC's operations *and* 0.92x of its cycles, plus width-parameterised helpers ASC cannot write
ASC	an instruction VMI cannot express is needed (a per-lane variable right shift), or a distribution mode is the point

Methodologically. Operation and line counts measure how much you write. They were a poor predictor of cycles here – PTO wins on 9 of 22 variants, and on four of those it *writes more operations than ASC*. Any frontend comparison reporting only source size reports half the answer.